

数据清洗与特征处理：

1. 缺失值处理：

观察 csv 文件中的数据，Age、Embarked、Cabin 这三列特征存在缺失值。对于这三个特征分析，需要结合各自的特性分析和处理：

Age：年龄通常用中位数填充，我在此处认为如果直接用中位数全部填充会造成年龄结构的改变，从统计角度考虑准确性——我采用了直接从已有年龄中随机抽取数值填充，这样可以保留原有年龄分布结构。（当然我还了解到可以更细分类别地填充中位数，如按 Sex + Pclass 分组填充，这样是考虑到了各性别或船舱等级的乘客年龄分布不同的因素。但是我觉得用在这有点繁琐遂没有采用。）

Embarked：采用最简洁的方式，使用众数填充。

Cabin：缺失太多，选择直接删除该特征列。

2. 数据转换：

把非数值型数据转化为数值型：

Sex：把男性和女性分别编为 0 和 1。

Embarked & Pclass：独热编码。（这里在过程中遇到了数值类型的问题，第一次输出是布尔型，在后面运算中出现报错；加上一行 dtype=int 转化为整数型。）

3. 特征工程：

特征选择：去掉了关联不大的 Ticket 和 Name 特征。***后来我发现 PassengerId 也需要去掉，此处截图未展示**

特征构造：**FamilySize：**把 SibSp 和 Parch 组合成家庭成员数量。

IsAlone：根据常识判断一个人出行很可能会提高存活率，所以单独构造一列增大对这个因素的考虑。

4. 标准化：划分数据集后对训练集和测试集的'Age','FamilySize','Fare'分别进行标准化，调整数据尺度便于模型更合理计算。

```
11 #导入数据
12 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
13 print(f"使用设备: {device}")
14 df = pd.read_csv("train.csv")
15
16 #数据预处理和特征工程
17 np.random.seed(42)
18 known_ages = df["Age"].dropna()
19 missing_age = df[df["Age"].isna()].index
20 df.loc[missing_age, "Age"] = np.random.choice(known_ages, size=len(missing_age))
21
22 df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])
23
24 df["Sex"] = df["Sex"].map({"male": 0, "female": 1})
25
26 df = df.drop(labels=["Cabin", "Ticket", "Name"], axis=1)
27
28 df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
29 df["IsAlone"] = (df["FamilySize"] == 1).astype(int)
30
31 pd.get_dummies(df, columns=["Embarked"], drop_first=True, dtype=int)
32 pd.get_dummies(df, columns=["Pclass"], drop_first=True, dtype=int)
```

划分数据集，设置 x, y 张量；

模型构造：

使用 torch.nn.Module 定义二分类模型，设置输入层、隐藏层、输出层；层内包含 Linear 线性层，归一化处理，ReLU 激活函数，随机 Dropout；定义前向传播。

损失函数使用 BCE 二元交叉熵损失。

优化器使用 Adam 优化器：根据梯度和轮数自我调节超参数优化，自适应 lr，收敛稳定。

主要超参数：每个 batch 内样本个数，隐藏层维度，Dropout rate，训练轮数。

```
64 class MLP(nn.Module): 1个用法
65     def __init__(self, input_dim):
66         super().__init__()
67
68         self.mlp = nn.Sequential(
69
70             nn.Linear(input_dim, out_features=128),
71             nn.BatchNorm1d(128),
72             nn.ReLU(),
73             nn.Dropout(0.2),
74
75             nn.Linear(in_features=128, out_features=32),
76             nn.BatchNorm1d(32),
77             nn.ReLU(),
78             nn.Dropout(0.2),
79
80             nn.Linear(in_features=32, out_features=16),
81             nn.ReLU(),
82
83             nn.Linear(in_features=16, out_features=1),
84         )
85
86     def forward(self, x):
87         return self.mlp(x)
88
```

```
132 #损失函数
133 criterion = nn.BCEWithLogitsLoss()
134 #优化器
135 optimizer = optim.Adam(
136     model.parameters(),
137     lr=0.0003
138 )
139
140 #训练
141 epochs = 1000
```

数据划分方式、随机种子和实验配置：

数据集随机划分：设置测试集比例，设置随机值，指定标签为 Survived。

```
41 train, test = train_test_split(  
42     *arrays: df,  
43     test_size=0.2,  
44     random_state=42,  
45     stratify=df["Survived"]  
46 )
```

设置随机种子，并应用到 Pytorch 和 Numpy 中。

```
17 seed = 42  
18  
19 np.random.seed(seed)  
20 torch.manual_seed(seed)
```

实验配置：我的电脑并没有独立显卡，所以使用 CPU 来运行；虚拟环境使用 anaconda 创建，使用的 Python 版本号为 3.10，其余可详见 requirements.txt。

遇到的问题：

1. 在第一次运行时发现测试集和训练集的准确率完全相同。检查后发现是 correct 和 total 的赋值有问题，测试集和训练集用了相同的变量名。最后我重新分别命名和计算，并且改变了 for 循环的结构，持续地同时监测训练过程中训练集和测试集的准确率及 loss 变化。

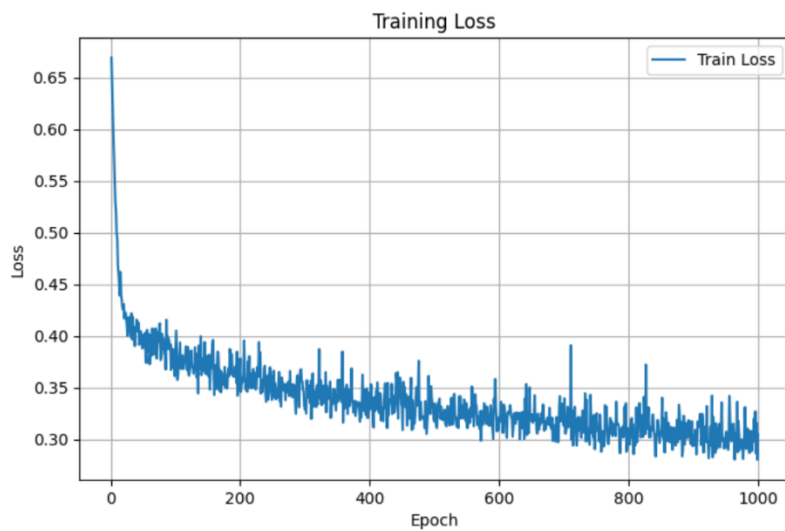
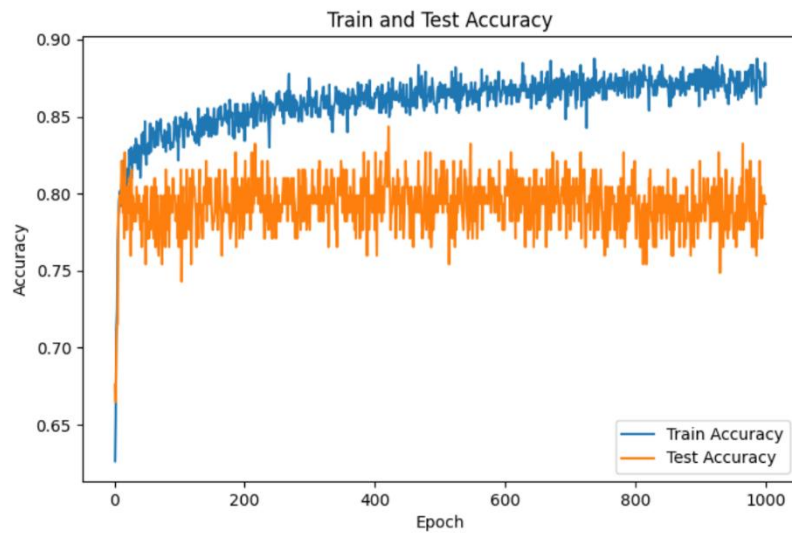
2. 对单个新乘客进行预测时，输出幸存概率一直是 100%。

最先我发现：不能对单个数据进行独热编码，否则会出现特征列与训练时不一致的情况；于是我把独热编码改为把特征手动填充为对应列，并对未填充列赋值为 0。

但此时仍旧输出 100%。

于是我多尝试输入了几个不同的乘客数据（控制变量法逐个改变着试），发现模型对票价特别敏感，100 的票价和 1000 的票价存活率差异巨大，之前一直输出 100%也极有可能是输入票价太高的缘故。输入较低票价时，概率就输出了看起来相对正常的值。（真的是金钱解决了一切问题吗！？）

3. 写代码过程中还遇到了一些小问题，比较繁琐，就不列出来了。



测试集准确率、实验结果分析和心得体会

使用设备：cpu

```
Epoch [100/1000], Loss: 0.3816, Train Acc: 0.8371Test Acc: 0.7933
Epoch [200/1000], Loss: 0.3783, Train Acc: 0.8343Test Acc: 0.8045
Epoch [300/1000], Loss: 0.3292, Train Acc: 0.8666Test Acc: 0.7765
Epoch [400/1000], Loss: 0.3457, Train Acc: 0.8652Test Acc: 0.7765
Epoch [500/1000], Loss: 0.3175, Train Acc: 0.8722Test Acc: 0.8101
Epoch [600/1000], Loss: 0.3214, Train Acc: 0.8652Test Acc: 0.8045
Epoch [700/1000], Loss: 0.3135, Train Acc: 0.8708Test Acc: 0.7765
Epoch [800/1000], Loss: 0.2983, Train Acc: 0.8750Test Acc: 0.8045
Epoch [900/1000], Loss: 0.3039, Train Acc: 0.8806Test Acc: 0.7933
Epoch [1000/1000], Loss: 0.3052, Train Acc: 0.8708Test Acc: 0.7933
```

进程已结束，退出代码为 0

训练集和测试集的准确率都较高且相近，个人认为模型效果不错。

心得体会：最开始学习的时候在网上找了很多课程，储备了一些理论但是不知道该怎么实际应用、比较迷茫；后来慢慢开始尝试写代码，有不太懂的和一些不太清楚的问题就询问了 AI，慢慢开始掌握，还是挺有成就感——把理论变成实践是一个很奇妙很有成就感的过程！写代码的过程中学到了很多，比如很多语法的逻辑、各个函数具体该怎样使用、各个步骤具体需要怎么放怎么安排结构才是最合理的，等等。很期待后续继续学习新东西！